CHAPTER 20

Monads and Effects

Count the question marks in the last Rust function you wrote. Each one says the same thing: "evaluate this; if it produced a value, continue with that value; otherwise stop here and hand the failure to my caller". You call `and_then` on an `Option` for the same reason. You call `flat_map` on an iterator when every element produces several results and you want all of them in one sequence. You write `.await` when a step may have to wait for the network before the rest of the function can run. Four spellings, in four corners of the language, of one pattern: *run a step that does something more than return a value, then feed its value to the rest of the computation.*

The pattern has a name and a theory. In Chapter 16 you saw that types and functions form a category. But a parser that may fail, a lookup that may find several answers, a step that updates a counter or waits for a socket: each of these has an **effect**, and functions with effects do not compose the way the category of types expects. At the end of the 1980s Eugenio Moggi proposed a uniform account of such "notions of computation" (his paper *Notions of computation and monads* appeared in 1991): an effectful function from A to B is a pure function from A to $M\,B$, where the type constructor M describes the effect, and a **monad** is exactly the structure that makes such functions compose again. Philip Wadler then showed functional programmers how to structure programs with the idea (*Monads for functional programming* is his best-known tutorial), and in the 1990s Haskell adopted monads to bring input and output into a pure language. Lean inherited the design.

This chapter, the last of Part IV, defines monads in three equivalent ways and proves that they agree. Then it follows the idea into both languages. Rust has many monads and no `Monad`: we will see exactly which features of Rust stand in the way (the missing higher-kinded types of Chapter 17, and the difference between `FnOnce` and `FnMut`), and what Rust does instead. Lean has a `Monad` class, `do` notation and monad transformers, and it lets us state and prove the laws that Rust can only test.

In this chapter you will learn:

- why functions with effects break "functions as morphisms", and how Kleisli arrows $A \to M\,B$ repair composition;
- the definition of a monad (`M`, `pure`, `bind`) with its three laws, and the equivalent definitions with `join` and with Kleisli composition;
- how `?`, `and_then`, `flat_map` and `.await` implement `bind` in Rust, and why Rust has no `Monad` trait;

- how Lean's `Monad` class and `do` notation work, what a `do` block desugars into, and what `LawfulMonad` and `IO` are;
- how to build a monad class with laws from scratch, prove the laws for `Option`, `List` and `State`, and build the Kleisli category of any monad on the `Category` class of Chapter 16;
- how monad transformers (`StateT`, `ExceptT`) combine effects, how Rust combines them ad hoc, and how adjunctions give rise to monads.

20.1 When functions have effects

Composition breaks

Here are two small functions from this chapter's main Rust example, `code/rust/examples/ch20_monads.rs`. The first parses a quantity typed by a user, the second checks that it is in range:

```rust
/// Parse a quantity: fails on text that is not a number.
fn parse_qty(s: &str) -> Option<u32> {
    s.trim().parse().ok()
}

/// Accept quantities between 1 and 100.
fn check_qty(n: u32) -> Option<u32> {
    if (1..=100).contains(&n) {
        Some(n)
    } else {
        None
    }
}
```

In the category of types and functions of Chapter 16, each is a morphism: `parse_qty` from `&str` to `Option<u32>`, `check_qty` from `u32` to `Option<u32>`. We would like to *compose* them, but composition requires the target of the first morphism to be the source of the second, and `Option<u32>` is not `u32`: `check_qty(parse_qty(s))` does not compile.

You know how to fix this. You write the glue by hand:

```rust
/// Kleisli composition for `Option`: run `f`, and if it produced a value,
/// run `g` on it.
fn compose_option<A, B, C>(
    f: impl Fn(A) -> Option<B>,
    g: impl Fn(B) -> Option<C>,
) -> impl Fn(A) -> Option<C> {
    move |a| f(a).and_then(|b| g(b))
}
```

`compose_option` runs `f`; if it produced a value, it runs `g` on it; if not, the composite fails. Look at what we did: we started reading `parse_qty` as a *partial* function from `&str` to `u32`, a function with an effect (it may fail), and defined a new way of composing such functions that takes care of the effect.

The same move works for every effect. Suppose M is a type constructor that describes an effect. A function with that effect from A to B is represented by a plain function

$$f : A \to M\,B$$

and such a function is called a **Kleisli arrow** from A to B (after the mathematician Heinrich Kleisli). Here are the effects this chapter meets, with the type constructor that describes each:

Effect	$M\,B$	Rust	Lean
may fail	$B + 1$	`Option<B>`	`Option B`
may fail with an error E	$B + E$	`Result<B, E>`	`Except E B`
has many possible results	lists of B	`Vec<B>`, iterators	`List B`
reads an environment R	$R \to B$	a `&R` parameter	`ReaderM R B`
reads and writes a state S	$S \to B \times S$	a `&mut S` parameter	`StateM S B`
writes a log W	$B \times W$	a `&mut Vec<_>` parameter	Mathlib's `Writer W B`
interacts with the world	(opaque)	any function at all	`IO B`
may suspend and resume	(opaque)	`impl Future<Output = B>`	(not in this book)

Two things stand out in the Rust column. Several entries are not type constructors at all: Rust handles state with `&mut`, and it does not record input and output in types, since any Rust function may print or open a file. And where Rust does have a type constructor, each one comes with its own syntax: `?`, `for`, `.await`. Both observations will matter.

Effects as typing rules

Moggi's idea can be stated as two typing rules. Distinguish *values* of type τ from *computations* of type $M\,\tau$, which may have the effect M when they run and then produce a τ. Two rules connect the two worlds:

$$\frac{\Gamma \vdash e : \tau}{\Gamma \vdash \mathit{pure}\, e : M\,\tau}\ \text{T-Pure} \qquad \frac{\Gamma \vdash e_1 : M\,\tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : M\,\tau_2}{\Gamma \vdash \texttt{let}\ x \Leftarrow e_1; e_2 : M\,\tau_2}\ \text{T-Bind}$$

Rule T-Pure says that a value can always be seen as a computation that has no effect. Rule T-Bind is rule T-Let of Chapter 1, for computations: run e_1, which may have its effect and then produces a τ_1; bind that result to x; continue with e_2, in which x is a plain τ_1. The whole is again a computation in M.

Notice what is missing: no rule takes you from $M\,\tau$ back to τ. You can use the result of a computation only *inside a larger computation of the same kind*. You know this discipline from Rust: once a step may fail, everything that depends on its result may fail too, unless you handle the failure explicitly. The arrow $\Leftarrow$ in T-Bind is spelled `let x = e1?;` in Rust for `Option` and `Result`, `let x = e1.await;` for futures, and `let x ← e1` in a Lean `do` block.

20.2 Monads: `pure`, `bind` and three laws

The definition

Definition. A **monad** consists of

- a type constructor M, which takes a type A to a type $M\,A$;
- for every type A, an operation $\mathit{pure} : A \to M\,A$;

- for all types A and B, an operation $\mathit{bind} : M\,A \to (A \to M\,B) \to M\,B$;

such that, for all values a, computations m and functions f and g of the appropriate types, three **laws** hold:

$$\mathit{bind}(\mathit{pure}\,a, f) = f\,a \qquad \text{(left identity)}$$

$$\mathit{bind}(m, \mathit{pure}) = m \qquad \text{(right identity)}$$

$$\mathit{bind}(\mathit{bind}(m, f), g) = \mathit{bind}(m, \lambda a.\ \mathit{bind}(f\,a, g)) \qquad \text{(associativity)}$$

`pure` and `bind` are rules T-Pure and T-Bind turned into functions. `pure a` is the computation that does nothing and returns `a`. `bind m f` runs `m`, passes its result to `f`, and runs the computation that `f` returns. For `Option`, `pure` is `Some` and `bind` is `and_then`. For `Vec`, `pure a` is `vec![a]` and `bind` is `flat_map` followed by `collect`. Mathematicians call this presentation a *Kleisli triple*; Haskell spells `bind m f` as `m >>= f`, and so does Lean.

The laws are easiest to understand as **refactorings that must not change what a program means**:

- *Left identity*: `let x ⇐ pure a; e` can be replaced by `e` with `a` for `x`. A step without an effect can be inlined.
- *Right identity*: `let x ⇐ m; pure x` can be replaced by `m`. A block that ends by returning the result of its last step unchanged *is* that last step.
- *Associativity*: given three steps, it does not matter whether you extract the first two or the last two into a helper function. "Extract function" is a safe refactoring.

The laws as category laws

The glue we wrote for `Option` works for any monad. **Kleisli composition** is defined, for $f : A \to M\,B$ and $g : B \to M\,C$, by

$$g \circ_M f = \lambda a.\ \mathit{bind}(f\,a, g) : A \to M\,C$$

Rewrite the three laws with it. Left identity says $g \circ_M \mathit{pure} = g$, right identity says $\mathit{pure} \circ_M f = f$, and associativity says

$$h \circ_M (g \circ_M f) = (h \circ_M g) \circ_M f$$

These are exactly the laws of a category from Chapter 16, with pure as the identity. (The names "left" and "right" cross over because $\circ$ is written right to left.) So we have found a new category:

> For every monad M, the **Kleisli category** of M has types as objects, Kleisli arrows $A \to M\,B$ as morphisms from A to B, pure as identities and $\circ_M$ as composition. The monad laws are the category laws.

Later in the chapter Lean checks this in both directions: the monad laws imply the category laws, and conversely.

`map` and `join`

Category theorists usually present a monad differently: as a functor with two natural transformations. Every monad is a functor in the sense of Chapter 17, because `map` can be defined from the other two operations:

$$\mathit{map}\ f\ m = \mathit{bind}(m, \lambda a.\ \mathit{pure}\,(f\ a))$$

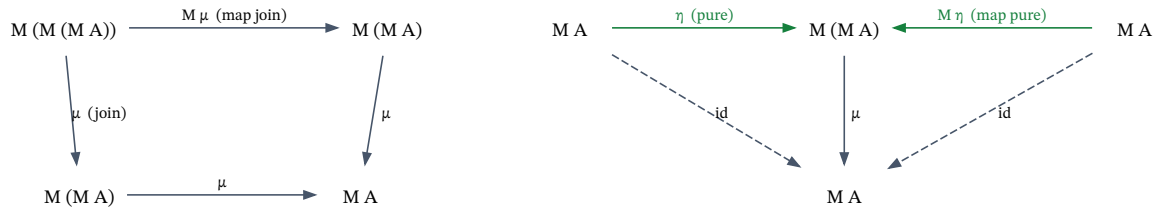
and the functor laws follow from the monad laws (we prove it in Lean below). Next to `pure`, which category theorists call η (eta, the *unit*), there is a flattening operation μ (mu, the *multiplication*), which programmers call `join`:

$$\mu_A : M\,(M\,A) \to M\,A \qquad\qquad \mu_A(mm) = \mathit{bind}(mm, \lambda m.\ m)$$

You have used it: `Option::flatten` turns an `Option<Option<T>>` into an `Option<T>`, and `Iterator::flatten` turns an iterator of iterators into one iterator. Both η and μ are natural transformations (Chapter 18), from the identity functor to M and from $M \circ M$ to M, and the monad laws become three equations between them:

$$\mu_A \circ M\mu_A = \mu_A \circ \mu_{MA} \qquad\qquad \mu_A \circ \eta_{MA} = \mathrm{id}_{MA} = \mu_A \circ M\eta_A$$

The first says that a computation with three layers can be flattened from the inside or from the outside with the same result. The other two say that adding a trivial layer, outside or inside, and flattening it again does nothing.

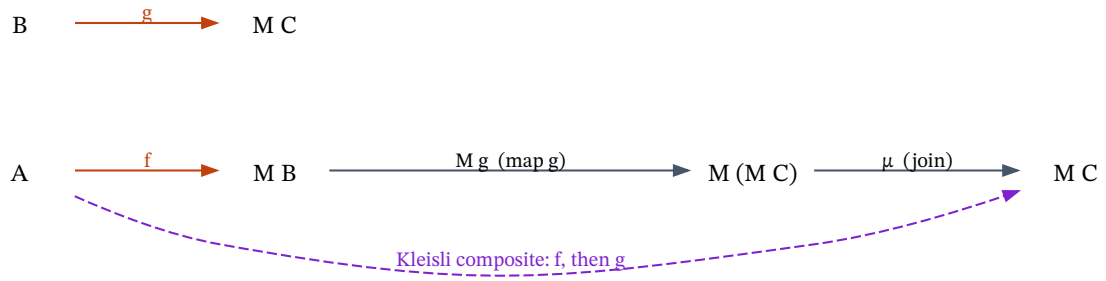


The monad laws in terms of join (μ) and pure (η). Left: flattening three layers from the outside or from the inside gives the same result. Right: adding a trivial layer with pure, outside (η) or inside (M η), and flattening again is the identity.

The two presentations are interchangeable. We just defined `map` and `join` from `bind`; conversely, `bind` is "map, then flatten":

$$\mathit{bind}(m, f) = \mu\,(\mathit{map}\ f\ m)$$

which is why Rust's `flat_map` has that name. In this language the Kleisli composite is $g \circ_M f = \mu_C \circ Mg \circ f$: apply f, map g over the result (which gives two layers of effects), and flatten.



Kleisli composition. Given $f : A \to M\,B$ and $g : B \to M\,C$, the composite runs f, maps g over its result to get $M\,(M\,C)$, and flattens with μ.

> **NOTE**
>
> The μ/η presentation explains a famous line from Saunders Mac Lane's *Categories for the Working Mathematician*: a monad is a monoid in the category of endofunctors. The "product" of two endofunctors is their composite; μ is a multiplication $M \circ M \to M$, η a unit $\mathrm{Id} \to M$, and the three equations above are the associativity and unit laws of a monoid (Chapter 14). You will see a down-to-earth instance of this below: the laws of the `Writer` monad are literally the monoid laws of its log.

20.3 Monads you already use in Rust

`?` is bind

Here is one function written three ways. All three parse a quantity, check it, and compute how many boxes it fills at two units per box:

```rust
/// With `?`: every `?` is a bind, and the rest of the function body is the
/// continuation that receives the value.
fn boxes_question(s: &str) -> Option<u32> {
    let n = parse_qty(s)?;
    let q = check_qty(n)?;
    Some(q.div_ceil(2))
}

/// With `and_then`: the continuations are written as closures.
fn boxes_and_then(s: &str) -> Option<u32> {
    parse_qty(s).and_then(|n| check_qty(n).and_then(|q| Some(q.div_ceil(2))))
}

/// With `match` and `return`: what `?` does on an `Option`, written out.
fn boxes_match(s: &str) -> Option<u32> {
    let n = match parse_qty(s) {
        Some(v) => v,
        None => return None,
    };
    let q = match check_qty(n) {
        Some(v) => v,
        None => return None,
    };
    Some(q.div_ceil(2))
}
```

The program checks that the three agree on every sample and prints the results of `compose_option(parse_qty, check_qty)` (as `read_qty`) and of the three versions (as `boxes`):

```
-- Option: ?, and_then and match agree --
      12: read_qty = Some(12), boxes = Some(6)
  twelve: read_qty = None, boxes = None
     500: read_qty = None, boxes = None
```

The `match` version is, conceptually, what the compiler does with `?` on an `Option`. Read `let n = parse_qty(s)?; REST` as `parse_qty(s).and_then(|n| REST)`, where `REST` is *the rest of the function body*: the continuation that receives the value. If there is no value, the continuation is skipped with an early `return`. That is `bind`, with an implicit continuation.

The real desugaring goes through two standard library traits. Roughly, `e?` becomes a `match` on `Try::branch(e)`, which returns `ControlFlow::Continue(v)` (continue with `v`) or `ControlFlow::Break(r)`, in which case the function returns `FromResidual::from_residual(r)`. Both traits are unstable (feature `try_trait_v2`), so on stable Rust `?` works only for the standard library's types (`Option`, `Result`, `ControlFlow` and `Poll` values wrapping them): `bind` for a fixed list of monads.

For `Result`, `?` does one more thing:

```rust
#[derive(Debug, PartialEq)]
enum OrderError {
    BadNumber(ParseIntError),
    OutOfRange(u32),
}

impl From<ParseIntError> for OrderError {
    fn from(e: ParseIntError) -> Self {
        OrderError::BadNumber(e)
    }
}

fn check_range(n: u32) -> Result<u32, OrderError> {
    if (1..=100).contains(&n) {
        Ok(n)
    } else {
        Err(OrderError::OutOfRange(n))
    }
}

/// `?` on a `Result`: bind, plus a conversion of the error through `From`.
fn boxes_result(s: &str) -> Result<u32, OrderError> {
    let n: u32 = s.trim().parse()?;
    let q = check_range(n)?;
    Ok(q.div_ceil(2))
}

/// The same function with the early returns and conversions written out.
fn boxes_result_desugared(s: &str) -> Result<u32, OrderError> {
    let n: u32 = match s.trim().parse::<u32>() {
        Ok(v) => v,
        Err(e) => return Err(From::from(e)),
    };
    let q = match check_range(n) {
        Ok(v) => v,
        Err(e) => return Err(From::from(e)),
    };
    Ok(q.div_ceil(2))
}
```

```
-- Result: ? converts errors with From --
      12: Ok(6)
  twelve: Err(BadNumber(ParseIntError { kind: InvalidDigit }))
     500: Err(OutOfRange(500))
```

The first `?` meets a `ParseIntError`, but the function returns `Result<u32, OrderError>`. `bind` for `Result<_, E>` would keep the error type; `?` converts it with `From::from`, hence our `From<ParseIntError>` impl. (For the second `?`, the blanket `impl<T> From<T> for T` makes the conversion the identity.) As a typing rule, a model in the style of Chapter 1, with *ret* the return type of the enclosing function:

$$\frac{\Gamma \vdash e : \texttt{Result}\langle \tau, E\rangle \qquad \mathit{ret} = \texttt{Result}\langle \sigma, F\rangle \qquad F : \texttt{From}\langle E\rangle}{\Gamma \vdash e? : \tau}\ \text{T-Try}$$

The conclusion gives `e?` the *unwrapped* type τ: that is how the rest of the function gets to see a plain value, as in rule T-Bind. And the premise about *ret* shows that the monad is fixed by the function's return type: one function body, one monad. Using `?` on an `Option` inside a function that returns a `Result` is rejected:

```rust
pub fn first_qty(lines: &[&str]) -> Result<u32, String> {
    let first = lines.first()?;
    first.parse::<u32>().map_err(|e| e.to_string())
}
```

```
error[E0277]: the `?` operator can only be used on `Result`s, not `Option`s, in a function
that returns `Result`
 --> compile_fail/ch20_question_mark_option.rs:6:30
  |
5 | pub fn first_qty(lines: &[&str]) -> Result<u32, String> {
  | ------------------------------------------------------- this function returns a `Result`
6 |     let first = lines.first()?;
  |                              ^ use `.ok_or(...)?` to provide an error compatible with
`Result<u32, String>`
```

The suggested fix, `.ok_or(...)?`, first translates the `Option` into a `Result`, a translation between monads that respects `pure` and `bind` (a *monad morphism*). Lean inserts such translations automatically, as we will see when we combine effects.

`and_then`, `flat_map` and iterator adapters

`Option::and_then` and `Result::and_then` are `bind` with an explicit closure. For sequences, `bind` is `Iterator::flat_map`: apply a function that returns many results to every element, and concatenate. Here it is at work on a nondeterministic question, "which warehouses and carriers could ship this item?":

```rust
/// Every warehouse that stocks `item`.
fn warehouses(item: &str) -> Vec<&'static str> {
    match item {
        "bolts" => vec!["Hamburg", "Lyon"],
        "nuts" => vec!["Lyon"],
        _ => vec![],
    }
}

/// Every carrier that collects from a warehouse.
fn carriers(site: &str) -> Vec<&'static str> {
```

```
    match site {
        "Hamburg" => vec!["DHL", "UPS"],
        "Lyon" => vec!["Geodis"],
        _ => vec![],
    }
}

/// `bind` for vectors: apply `f` to every element and concatenate.
fn bind_vec<A, B>(m: Vec<A>, f: impl FnMut(A) -> Vec<B>) -> Vec<B> {
    m.into_iter().flat_map(f).collect()
}

/// Every (warehouse, carrier) pair that can ship `item`.
fn shipping_options(item: &str) -> Vec<(&'static str, &'static str)> {
    bind_vec(warehouses(item), |w| {
        bind_vec(carriers(w), |c| vec![(w, c)])
    })
}
```

```
-- Vec: flat_map is bind --
   bolts: [("Hamburg", "DHL"), ("Hamburg", "UPS"), ("Lyon", "Geodis")]
    nuts: [("Lyon", "Geodis")]
  screws: []
```

Read `shipping_options` as T-Bind twice: choose a warehouse `w`, then a carrier `c` for it, then return the pair. The list monad explores every choice. An empty list means "no possible result", so `Vec` generalizes `Option`, which is a list with at most one element.

Other iterator adapters are monad operations in disguise: `flatten` is `join`, and `iter.collect::<Option<Vec<_>>>()` turns a sequence of computations that may fail into one computation that fails if any of them does, an operation we will meet again as `sequence`.

The example also checks the three laws for `Option` and `Vec` on samples:

```
/// The three monad laws, checked on samples: for `Option` (pure = `Some`,
/// bind = `and_then`) and for `Vec` (pure = `vec![a]`, bind = `flat_map`).
fn check_laws() {
    for s in ["12", "twelve", "500", "0", " 7 "] {
        let m = parse_qty(s);
        // Left identity: pure(a) >>= f  ==  f(a)
        assert_eq!(Some(s).and_then(parse_qty), parse_qty(s));
        // Right identity: m >>= pure  ==  m
        assert_eq!(m.and_then(Some), m);
        // Associativity: (m >>= f) >>= g  ==  m >>= (|x| f(x) >>= g)
        let half = |n: u32| n.checked_sub(1).map(|k| k / 2);
        assert_eq!(
            m.and_then(check_qty).and_then(half),
            m.and_then(|n| check_qty(n).and_then(half))
        );
    }
    for item in ["bolts", "nuts", "screws"] {
        let m = warehouses(item);
        assert_eq!(bind_vec(vec![item], warehouses), warehouses(item));
        assert_eq!(bind_vec(m.clone(), |x| vec![x]), m);
        assert_eq!(
            bind_vec(bind_vec(m.clone(), carriers), services),
            bind_vec(m.clone(), |w| bind_vec(carriers(w), services))
        );
    }
}
```

```
-- the monad laws hold on every sample --
```

As in Chapter 14, a Rust test can confirm the laws on the inputs we thought of and nothing more. The Lean instances below prove them for all inputs.

State, the hard way and the Rust way

`Option`, `Result` and `Vec` look like containers. The state monad does not: a computation in it is a *function* from the current state to a result and a new state. We can build it in Rust with closures:

```
/// A stateful computation: a function from the current state to a result and
/// a new state. It is boxed because every closure has its own type, and
/// `bind` must return one type, whatever closure it was given.
struct State<'a, S, A>(Box<dyn FnOnce(S) -> (A, S) + 'a>);

impl<'a, S: 'a, A: 'a> State<'a, S, A> {
    fn pure(a: A) -> Self {
        State(Box::new(move |s| (a, s)))
    }

    fn bind<B: 'a>(self, f: impl FnOnce(A) -> State<'a, S, B> + 'a) -> State<'a, S, B> {
        State(Box::new(move |s| {
            let (a, s1) = (self.0)(s);
            (f(a).0)(s1)
        }))
    }

    fn run(self, initial: S) -> (A, S) {
        (self.0)(initial)
    }
}

/// Read the state.
fn get<'a, S: Clone + 'a>() -> State<'a, S, S> {
    State(Box::new(|s: S| (s.clone(), s)))
}

/// Replace the state.
fn put<'a, S: 'a>(new: S) -> State<'a, S, ()> {
    State(Box::new(move |_| ((), new)))
}
```

Each closure has its own anonymous type (Chapter 8), and `bind` must return a single type whatever closures it was given, so the closure goes into a `Box<dyn FnOnce(S) -> (A, S)>`, an existential package (Chapter 15); `+ 'a` says that what it captures must live at least as long as `'a`. `bind` builds a closure that runs `self`, feeds the result to `f`, and runs the computation `f` returns from the intermediate state `s1`. Here is a program that gives every order line a fresh ticket number, next to what a Rust programmer would write:

```
/// A fresh ticket number: read the counter, advance it, return the old value.
fn fresh<'a>() -> State<'a, u32, u32> {
    get().bind(|n| put(n + 1).bind(move |()| State::pure(n)))
}

/// Give every order line a ticket number, as `label` does in `State.lean`.
fn label<'a>(items: &'a [&'a str]) -> State<'a, u32, Vec<(u32, &'a str)>> {
    match items.split_first() {
        None => State::pure(Vec::new()),
        Some((first, rest)) => fresh().bind(move |n| {
            label(rest).bind(move |tail| {
                let mut lines = vec![(n, *first)];
                lines.extend(tail);
                State::pure(lines)
            })
        }),
```

```
    }
}

/// What a Rust programmer writes instead: the state is a `&mut` borrow.
fn label_mut<'a>(items: &[&'a str], counter: &mut u32) -> Vec<(u32, &'a str)> {
    let mut lines = Vec::new();
    for item in items {
        lines.push((*counter, *item));
        *counter += 1;
    }
    lines
}
```

```
-- State: a monad of closures, and &mut --
label     : [(100, "bolts"), (101, "nuts"), (102, "washers")], counter = 103
label_mut : same lines, counter = 103
```

Both versions produce the same lines and leave the counter at 103. The monadic one allocates a box for every `bind`, needs lifetime annotations, and reads inside out; the `&mut` one is short, fast and obvious. The state monad is how languages without mutation express state. Rust has *controlled* mutation instead: a `&mut u32` is a unique, temporary permission to change the counter, and the borrow checker (Part VI) guarantees that nobody else observes the counter meanwhile. Threading the state through a function $S \to A \times S$ and lending a unique mutable reference are two answers to the same question. Likewise, the reader monad is an extra `&Config` parameter in Rust, and the writer monad a `&mut Vec<String>` parameter.

20.4 async and await: a monad built into the compiler

A future is a computation that may have to wait. The `Future` trait has one method, `poll`, which returns `Poll::Ready(value)` or `Poll::Pending` ("not yet, poll me again later"). Something has to keep calling `poll`: an **executor**. `code/rust/examples/ch20_async.rs` contains the smallest executor that works with the standard library alone:

```
/// A minimal executor: poll the future until it is ready, and report how many
/// polls it took. A real executor sleeps until a waker fires; ours retries at
/// once, which is enough for futures that never wait for the outside world.
fn block_on<F: Future>(fut: F) -> (F::Output, u32) {
    let mut fut = pin!(fut);
    let mut cx = Context::from_waker(Waker::noop());
    let mut polls = 0;
    loop {
        polls += 1;
        if let Poll::Ready(value) = fut.as_mut().poll(&mut cx) {
            return (value, polls);
        }
    }
}
```

`pin!` fixes the future in place on the stack (a future may hold references into itself, so it must not move once polled), and `Waker::noop()` is a waker that does nothing, enough for an executor that polls again at once. Here is a future that is not ready the first time, standing in for a network round trip:

```
/// A future that is not ready the first time it is polled, standing in for a
/// network round trip.
struct NetworkDelay {
```

```
    waited: bool,
}

impl Future for NetworkDelay {
    type Output = ();

    fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<()> {
        if self.waited {
            Poll::Ready(())
        } else {
            self.waited = true;
            cx.waker().wake_by_ref(); // "poll me again"
            Poll::Pending
        }
    }
}

fn network_delay() -> NetworkDelay {
    NetworkDelay { waited: false }
}
```

And here is a price quote that fetches prices from a simulated service:

```
#[derive(Debug, PartialEq)]
enum ShopError {
    UnknownItem(String),
}

/// Ask the (simulated) price service for the unit price of an item.
async fn fetch_price(item: &str) -> Result<u32, ShopError> {
    network_delay().await;
    match item {
        "bolts" => Ok(25),
        "nuts" => Ok(10),
        _ => Err(ShopError::UnknownItem(item.to_string())),
    }
}

/// Two effects in one function: `.await` may suspend, `?` may fail. Each is a
/// bind, and the compiler supports both.
async fn quote(lines: &[(&str, u32)]) -> Result<u32, ShopError> {
    let mut total = 0;
    for &(item, qty) in lines {
        let price = fetch_price(item).await?;
        total += price * qty;
    }
    Ok(total)
}
```

```
quote([("bolts", 2), ("nuts", 3)]) = Ok(80) after 3 polls
quote([("bolts", 2), ("screws", 1), ("nuts", 3)]) = Err(UnknownItem("screws")) after 3 polls
  (stopped at "screws": the line after it was never priced)
and_then(ready(20), |n| ready(n + 1)) = 21 after 1 poll
```

Each `fetch_price` waits once, so the first quote needs three polls: two that stop at a `network_delay`, and one that finishes. The second quote stops at `"screws"`: the `?` returns the error and the nuts are never priced.

In monadic terms, `impl Future<Output = B>` is a computation $M\,B$, `.await` is `bind`, an `async` block is a block of T-Bind steps, `std::future::ready` is `pure`, and the executor is what *runs* the monad, as `run` did for our `State`. We can even write `bind` for futures with `async` itself:

```
/// `bind` for futures, written with `async` itself: run `first`, feed its
/// output to `f`, run the future that `f` returns.
```

```
fn and_then<A, B, Fut1, Fut2>(
    first: Fut1,
    f: impl FnOnce(A) -> Fut2,
) -> impl Future<Output = B>
where
    Fut1: Future<Output = A>,
    Fut2: Future<Output = B>,
{
    async move {
        let a = first.await;
        f(a).await
    }
}
```

The last line of the output shows `and_then(ready(20), |n| ready(n + 1))` completing at the first poll, as left identity predicts. (The program also checks left identity on `fetch_price`.)

The compiler does not implement `async` with closures and `bind`, though. It turns the body of an `async fn` into a state machine, roughly an `enum` with a variant for every `.await` at which the function can be suspended, holding the local variables alive at that point. That is how `quote` can update `total` in a `for` loop across suspension points. And notice `.await?`: `quote` has *two* effects, suspension and failure, stacked on top of each other. Lean needs a library construction for that, monad transformers; Rust builds it into the syntax.

20.5 Why Rust has no `Monad` trait

Rust has all these monads, but no trait that they share, and no way to write a function once for all of them. There are three reasons, and the second is the most interesting.

No higher-kinded types

A `Monad` trait would have to abstract over `Option` itself, a type constructor of kind $Type \to Type$, but Rust's traits abstract over types. Chapter 17 met this obstacle for `Functor`, and the workaround: a "family" trait, in which a marker type stands for the type constructor and a generic associated type applies it. The monad version is in `code/rust/examples/ch20_monad_trait.rs`:

```
/// A family of types with `pure` and `bind`. `Self` is a marker type that
/// stands for a type constructor; `Self::Of<T>` applies it to `T`.
trait Monad {
    type Of<T>;

    fn pure<A>(a: A) -> Self::Of<A>;

    /// `FnMut`, not `FnOnce`: the `Vec` instance calls `f` once per element.
    fn bind<A, B, F>(m: Self::Of<A>, f: F) -> Self::Of<B>
    where
        F: FnMut(A) -> Self::Of<B>;
}

/// The marker for `Option`.
struct OptionM;

/// The marker for `Vec`.
struct VecM;

impl Monad for OptionM {
    type Of<T> = Option<T>;
```

```
    fn pure<A>(a: A) -> Option<A> {
        Some(a)
    }

    fn bind<A, B, F>(m: Option<A>, f: F) -> Option<B>
    where
        F: FnMut(A) -> Option<B>,
    {
        m.and_then(f)
    }
}

impl Monad for VecM {
    type Of<T> = Vec<T>;

    fn pure<A>(a: A) -> Vec<A> {
        vec![a]
    }

    fn bind<A, B, F>(m: Vec<A>, f: F) -> Vec<B>
    where
        F: FnMut(A) -> Vec<B>,
    {
        m.into_iter().flat_map(f).collect()
    }
}
```

With it we can finally write code once for every monad:

```
/// Kleisli composition, once for every monad.
fn kleisli<M: Monad, A, B, C>(
    mut f: impl FnMut(A) -> M::Of<B>,
    g: impl FnMut(B) -> M::Of<C> + Clone,
) -> impl FnMut(A) -> M::Of<C> {
    move |a| M::bind(f(a), g.clone())
}

/// Every combination of a result of `ma` with a result of `mb`.
fn pair<M: Monad, A: Clone, B>(ma: M::Of<A>, mb: M::Of<B>) -> M::Of<(A, B)>
where
    M::Of<B>: Clone,
{
    M::bind(ma, move |a: A| {
        M::bind(mb.clone(), move |b: B| M::pure((a.clone(), b)))
    })
}

/// Turn a list of computations into one computation of a list.
fn sequence<M: Monad, A: Clone>(ms: Vec<M::Of<A>>) -> M::Of<Vec<A>>
where
    M::Of<A>: Clone,
{
    let mut acc = M::pure(Vec::new());
    for m in ms {
        acc = M::bind(acc, move |done: Vec<A>| {
            M::bind(m.clone(), move |x: A| {
                let mut next = done.clone();
                next.push(x);
                M::pure(next)
            })
        });
    }
    acc
}
```

`pair` combines two computations into a computation of pairs, and `sequence` turns a list of computations into a computation of a list. The same code means very different things in different monads:

```
// The same generic functions, used with two different monads. The marker
// type must be named: the compiler cannot work out `M` from `Option<u32>`.
let mut read_qty = kleisli::<OptionM, _, _, _>(parse_qty, check_qty);
let mut variants = kleisli::<VecM, _, _, _>(sizes, colors);
println!("read_qty(\"12\")  = {:?}", read_qty("12"));
println!("read_qty(\"500\") = {:?}", read_qty("500"));
println!("variants(\"tee\") = {:?}", variants("tee"));

println!(
    "pair, Option  = {:?}",
    pair::<OptionM, _, _>(Some(3), Some("bolts"))
);
println!(
    "pair, Vec     = {:?}",
    pair::<VecM, _, _>(vec![1, 2], vec!['a', 'b'])
);

let all = sequence::<OptionM, _>(vec![Some(1), Some(2), Some(3)]);
let one_missing = sequence::<OptionM, _>(vec![Some(1), None, Some(3)]);
let grid = sequence::<VecM, _>(vec![vec!["S", "M"], vec!["red", "blue"]]);
println!("sequence, Option = {all:?} and {one_missing:?}");
println!("sequence, Vec    = {grid:?}");
```

```
read_qty("12")  = Some(12)
read_qty("500") = None
variants("tee") = ["tee/S/red", "tee/S/blue", "tee/M/red", "tee/M/blue"]
pair, Option  = Some((3, "bolts"))
pair, Vec     = [(1, 'a'), (1, 'b'), (2, 'a'), (2, 'b')]
sequence, Option = Some([1, 2, 3]) and None
sequence, Vec    = [["S", "red"], ["S", "blue"], ["M", "red"], ["M", "blue"]]
```

For `Option`, `pair` succeeds only if both sides do, and `sequence` is all-or-nothing, like `collect::<Option<Vec<_>>>()`. For `Vec`, `pair` is the cartesian product and `sequence` enumerates every combination of choices. One definition, two effects: that is what an abstraction over monads buys.

It comes at a price, visible in every call: the turbofish `::<OptionM, ...>`. Leave it out, and the compiler gives up:

```
trait Monad {
    type Of<T>;
    fn pure<A>(a: A) -> Self::Of<A>;
}

struct OptionM;

impl Monad for OptionM {
    type Of<T> = Option<T>;
    fn pure<A>(a: A) -> Option<A> {
        Some(a)
    }
}

fn twice<M: Monad>(n: u32) -> M::Of<u32> {
    M::pure(n * 2)
}

pub fn answer() -> Option<u32> {
    twice(21)
}
```

```
error[E0283]: type annotations needed
  --> compile_fail/ch20_monad_family_inference.rs:25:5
   |
25 |     twice(21)
```

```
   |     ^^^^^ cannot infer type of the type parameter `M` declared on the function `twice`
   |
   = note: the type must implement `Monad`
help: the trait `Monad` is implemented for `OptionM`
  --> compile_fail/ch20_monad_family_inference.rs:13:1
   |
13 | impl Monad for OptionM {
   | ^^^^^^^^^^^^^^^^^^^^^^
   = note: cannot satisfy `<_ as Monad>::Of<u32> == Option<u32>`
note: required by a bound in `twice`
  --> compile_fail/ch20_monad_family_inference.rs:20:13
   |
20 | fn twice<M: Monad>(n: u32) -> M::Of<u32> {
   |             ^^^^^ required by this bound in `twice`
help: consider specifying a concrete type for the type parameter `M`
   |
25 |     twice::</* Type */>(21)
   |          ++++++++++++++
```

The note is the heart of the matter: the compiler must solve `<_ as Monad>::Of<u32> == Option<u32>` for the blank, and it will not, because two different markers could both declare `type Of<T> = Option<T>`. A real higher-kinded parameter `M` with `M<u32> = Option<u32>` would determine `M = Option` by unification (Chapter 13); a type family does not.

`FnOnce` or `FnMut`?

Look again at the closure bounds. `Option::and_then` takes an `FnOnce`, while `Iterator::flat_map` takes an `FnMut`. This is no accident: `bind` for `Option` calls its continuation at most once, `bind` for `Vec` once per element. In Chapter 8 the three closure traits answered the question "how many times may this closure be called?", and the answer differs between the two monads.

A single `Monad` trait has to choose one bound. If it chooses `FnOnce`, the `Vec` instance cannot be written:

```
pub fn bind_vec<A, B, F: FnOnce(A) -> Vec<B>>(m: Vec<A>, f: F) -> Vec<B> {
    let mut out = Vec::new();
    for a in m {
        out.extend(f(a));
    }
    out
}
```

```
error[E0382]: use of moved value: `f`
 --> compile_fail/ch20_vec_bind_fnonce.rs:8:20
  |
5 | pub fn bind_vec<A, B, F: FnOnce(A) -> Vec<B>>(m: Vec<A>, f: F) -> Vec<B> {
  |                                                          - move occurs because `f` has
type `F`, which does not implement the `Copy` trait
6 |     let mut out = Vec::new();
7 |     for a in m {
  |     ---------- inside of this loop
8 |         out.extend(f(a));
  |                    ^---
  |                    |
  |                    `f` moved due to this call, in previous iteration of loop
  |
note: `FnOnce` closures can only be called once
 --> compile_fail/ch20_vec_bind_fnonce.rs:5:26
  |
5 | pub fn bind_vec<A, B, F: FnOnce(A) -> Vec<B>>(m: Vec<A>, f: F) -> Vec<B> {
  |                          ^^^^^^^^^^^^^^^^^^^ `F` is made to be an `FnOnce` closure here
...
8 |         out.extend(f(a));
```

```
  |                    ---- this value implements `FnOnce`, which causes it to be moved when
called
```

Calling an `FnOnce` consumes it, so the second iteration of the loop would use a moved value. This is error E0382, the same error as using a moved `String` twice: in Part V's terms, an `FnOnce` closure is a resource that can be used once, and calling it twice would need contraction. If the trait chooses `FnMut`, as ours does, then `Vec` is fine, but every `Option` user pays:

```
fn bind_option<A, B, F>(m: Option<A>, mut f: F) -> Option<B>
where
    F: FnMut(A) -> Option<B>,
{
    match m {
        Some(a) => f(a),
        None => None,
    }
}

pub fn ticket(id: Option<u32>, customer: String) -> Option<(u32, String)> {
    bind_option(id, move |n| Some((n, customer)))
}
```

```
error[E0507]: cannot move out of `customer`, a captured variable in an `FnMut` closure
  --> compile_fail/ch20_option_bind_fnmut.rs:17:39
   |
16 | pub fn ticket(id: Option<u32>, customer: String) -> Option<(u32, String)> {
   |                                --------  ------ move occurs because `customer` has type
`String`, which does not implement the `Copy` trait
   |                                |
   |                                captured outer variable
17 |     bind_option(id, move |n| Some((n, customer)))
   |                     --------          ^^^^^^^^ `customer` is moved here
   |                     |
   |                     captured by this `FnMut` closure
   |
help: `Fn` and `FnMut` closures require captured values to be able to be consumed multiple
times, but `FnOnce` closures may consume them only once
  --> compile_fail/ch20_option_bind_fnmut.rs:8:8
   |
 8 |     F: FnMut(A) -> Option<B>,
   |        ^^^^^^^^^^^^^^^^^^^^^
```

The closure gives away the `String` it captured. That is fine for a closure that runs once, and `id.and_then(move |n| Some((n, customer)))` compiles. But an `FnMut` closure must be ready to run again, and a second run would have no `customer` left to give away, as the help message explains. The same cost appears in our generic functions: `pair` and `sequence` need `Clone` bounds and call `clone()` inside their closures, copies that `Option` never needs.

In Haskell or Lean every value can be used as often as you like, so the question "how many times will `bind` run the continuation?" never comes up, and one `Monad` class covers both cases. In Rust the question is part of the type, because closures own what they capture. The textbook definition of a monad does not ask how often a function $A \to M\ B$ will be called; Rust's ownership discipline makes that a real difference. It is Part V's resource reading of types, showing up far from ownership itself.

Control flow and borrowing

The third reason is practical. `?` and `.await` are not closures: they can sit inside a loop or a `match` arm, `return` from the function, and hold a mutable borrow of a local variable across several steps, as

`quote` does with `total`. A closure passed to `and_then` cannot `return` from the enclosing function or `break` out of its loop, and a closure that captures `&mut total` conflicts with every other use of `total` while it exists.

So Rust took a different road. It has the *instances* (`Option`, `Result`, iterators, futures) and *dedicated syntax* for the most important ones (`?`, `for`, `.await`), each zero-cost and each composing with the others (`.await?` works). Other effects are expressed with `&mut`, with parameters, or not at all (input and output). What Rust gives up is the abstraction: there is no `sequence` for every monad, and no way to give a new effect its own `?` on stable Rust.

20.6 Monads in Lean

The `Monad` class and `do` notation

Lean's core library defines monads with a small hierarchy of type classes (Chapter 14). The two operations come from the classes `Pure` and `Bind`, and `Monad` extends both (through `Applicative`, which this chapter does not need). The code is in `code/lean/Book/Ch20/DoNotation.lean`:

```
/-- info: @pure : {f : Type u_1 → Type u_2} → [self : Pure f] → {α : Type u_1} → α → f α -/
#guard_msgs in
#check @pure

/--
info: @bind : {m : Type u_1 → Type u_2} → [self : Bind m] → {α β : Type u_1} → m α → (α → m
β) → m β
-/
#guard_msgs in
#check @bind
```

Read the second signature with all implicit arguments shown: for any type constructor `m` (`u_1` and `u_2` are universe variables, Chapter 12) that has a `Bind` instance (the square brackets mark an instance argument), and any types `α` and `β`, `bind` takes an `m α` and a function `α → m β` and returns an `m β`: the definition of a monad, minus the laws. `x >>= f` is notation for `bind x f`. The core library has instances for `Option`, `Except ε`, `Id`, the state and reader monads, `IO` and more, but none for `List`; Mathlib adds one in `Mathlib.Data.List.Monad`.

As you saw in Chapter 2, monadic code in Lean is usually written with `do` notation. Here is the boxes computation of the Rust example:

```
def parseQty (s : String) : Option Nat := s.toNat?

def checkQty (n : Nat) : Option Nat := if 1 ≤ n ∧ n ≤ 100 then some n else none

/-- Parse, check, and compute the number of boxes (two units per box). -/
def boxes (s : String) : Option Nat := do
  let n ← parseQty s
  let q ← checkQty n
  return (q + 1) / 2

/-- info: [some 6, none, none] -/
#guard_msgs in
#eval ["12", "twelve", "500"].map boxes
```

`let n ← parseQty s` is rule T-Bind: run `parseQty s`, and if it produces a value, call it `n`. `return` is `pure`. To see what Lean makes of the block, print the definition with notation switched off (`set_option pp.notation false in` turns off all notation for the next command, so that `>>=`, `do` and even `+` and `/` are shown as plain function applications):

```
/--
info: def Ch20.Do.boxes : String → Option Nat :=
fun s => bind (parseQty s) fun n => bind (checkQty n) fun q => pure (HDiv.hDiv (HAdd.hAdd q
1) 2)
-/
#guard_msgs in
set_option pp.notation false in
#print boxes

/-- The `do` block and the chain of binds are the same term. -/
example (s : String) :
    boxes s = (parseQty s >>= fun n => checkQty n >>= fun q => pure ((q + 1) / 2)) :=
  rfl
```

There is no magic. Each `let x ← e` becomes `bind e (fun x => ...)`, with the rest of the block as the body of the function: the continuation, exactly as for Rust's `?`. The `example` confirms that the `do` block and the hand-written chain of `>>=` are the same term (`rfl` proves it). Nested actions follow the same scheme: `(← e)` inside an expression means "run `e` first, and use its result here":

```
/-- `(← e)` runs `e` first and uses its result in place. -/
def totalQty (a b : String) : Option Nat := do
  return (← parseQty a) + (← parseQty b)

/--
info: def Ch20.Do.totalQty : String → String → Option Nat :=
fun a b =>
  bind (parseQty a) fun __do_lift => bind (parseQty b) fun __do_lift_1 => pure (HAdd.hAdd
__do_lift __do_lift_1)
-/
#guard_msgs in
set_option pp.notation false in
#print totalQty
```

The elaborator named the two results `__do_lift` and `__do_lift_1` and moved both binds in front of the addition, left to right. The other constructs of `do` desugar in the same spirit: a line `e` whose result is unused becomes `bind e (fun _ => ...)`, `let x := e` is an ordinary `let`, and the `let mut` variables and `for` loops of Chapter 2 become state passed from one iteration to the next.

`LawfulMonad`

Lean's `Monad` class does not mention the laws, so an instance may break them. A separate `Prop`-valued class, `LawfulMonad m`, collects them as proof obligations, like the `LawfulMonoid` of Chapter 14. Its fields include `pure_bind` (`pure x >>= f = f x`) and `bind_assoc`, together with laws relating `bind` to `map` and `<*>` from `Functor` and `Applicative`. Right identity is not a field but a theorem, `bind_pure`, derived from the others. For a lawful monad the laws become rewriting rules for `do` blocks, proved here for an arbitrary monad `m`:

```
section Laws

variable {m : Type → Type} [Monad m] [LawfulMonad m] {α β γ : Type}

/-- Left identity: a `pure` step can be inlined. -/
```

```
example (a : α) (f : α → m β) : (do let x ← pure a; f x) = f a := by
  simp only [pure_bind]

/-- Right identity: binding into `pure` changes nothing. -/
example (x : m α) : (x >>= fun a => pure a) = x := by
  simp only [bind_pure]

/-- The `do` elaborator applies right identity by itself: this block *is* `x`. -/
example (x : m α) : (do let a ← x; pure a) = x := rfl

/-- Associativity: extracting a sub-block into a helper (or inlining it) does
not change the program. -/
example (x : m α) (f : α → m β) (g : β → m γ) :
    (do let b ← (do let a ← x; f a); g b) = (do let a ← x; let b ← f a; g b) := by
  simp only [bind_assoc]

end Laws
```

`simp only [l]` rewrites the goal with the lemma `l` and closes it if both sides become equal. The third `example` is a curiosity worth knowing: the `do` elaborator *itself* applies right identity, turning `let a ← x; pure a` into plain `x`, so `rfl` proves the equation. The language relies on a law that the `Monad` class does not require. The core library's monads are lawful:

```
example : LawfulMonad Option := inferInstance
example : LawfulMonad (Except String) := inferInstance
example : LawfulMonad (StateM Nat) := inferInstance
```

`IO`

`IO α` is the type of computations that may interact with the outside world and then produce an `α`. Defining one does nothing; running it is a separate step, performed by the Lean runtime when the action is a program's `main`, or by `#eval` in the editor:

```
/-- An `IO` action: a description of a computation that interacts with the
world. Defining it prints nothing. -/
def greet (name : String) : IO Unit := do
  IO.println s!"Hello, {name}."
  IO.println "Your order has shipped."

/--
info: Hello, Ada.
Your order has shipped.
-/
#guard_msgs in
#eval greet "Ada"
```

Because actions are values, you can store them and decide later in which order to run them. Here a list of three actions is built (printing nothing), then run in reverse; each action also increments a mutable reference created with `IO.mkRef`:

```
/-- Actions are values: we can put them in a list, reorder them, and run them
later. A mutable reference (`IO.mkRef`) is an effect of `IO` too. -/
def countdown : IO Nat := do
  let counter ← IO.mkRef 0
  let steps : List (IO Unit) :=
    [1, 2, 3].map fun i => do
      IO.println s!"step {i}"
      counter.modify (· + 1)
  for step in steps.reverse do
    step
  counter.get
```

```
/--
info: step 3
step 2
step 1
---
info: 3
-/
#guard_msgs in
#eval countdown
```

The messages come out in the order the actions were *run*, not the order in which they were *built*, and the final `3` is the value of the whole action. A Rust function call performs its effects; a Lean `IO` value only describes them.

What is an `IO` action, then? Lean's own definitions answer:

```
/-- info: @[reducible] def IO : Type → Type :=
EIO IO.Error -/
#guard_msgs in
#print IO

/-- info: def EIO : Type → Type → Type :=
fun ε α => EST ε IO.RealWorld α -/
#guard_msgs in
#print EIO

/-- info: def EST : Type → Type → Type → Type :=
fun ε σ α => Void σ → EST.Out ε σ α -/
#guard_msgs in
#print EST
```

`IO` is `EIO IO.Error`, `EIO ε` is `EST ε IO.RealWorld`, and an `EST ε σ α` is a function that takes a token of type `Void σ` and returns an `EST.Out ε σ α`: a result or an error, each paired with a new token. So `IO` is a state monad with exceptions, whose "state" is a token standing for the whole world (`IO.RealWorld`). The token carries no data; passing it from step to step forces the steps to happen in order. (This `Void σ` is an opaque token type, unrelated to the empty `Void` of Chapter 10.) The most impure-looking monad of all is, in its type, the state monad you are about to build, plus errors.

20.7 A monad class from scratch

Lean's `Monad` and `LawfulMonad` split the operations from the laws and carry extra structure. To see the essentials, we build our own class, with the laws as fields, in `code/lean/Book/Ch20/Monad.lean`. Inside `namespace Ch20` the name `Monad` refers to this class, which shadows Lean's (still available as `_root_.Monad`); that file does not use `do` notation, so we write `Monad.bind` and `Monad.pure` explicitly.

The class and three instances

```
/-- A monad: a type constructor `M` with two operations and three laws. -/
class Monad (M : Type → Type) where
  /-- Wrap a plain value in a computation that has no effect. -/
  pure : {α : Type} → α → M α
  /-- Run `m`, then feed its result to `f`. -/
  bind : {α β : Type} → M α → (α → M β) → M β
  /-- Left identity: binding a pure value is just function application. -/
  pure_bind : ∀ {α β : Type} (a : α) (f : α → M β), bind (pure a) f = f a
  /-- Right identity: binding into `pure` changes nothing. -/
```

```
  bind_pure : ∀ {α : Type} (m : M α), bind m pure = m
  /-- Associativity: only the order of the steps matters, not their nesting. -/
  bind_assoc : ∀ {α β γ : Type} (m : M α) (f : α → M β) (g : β → M γ),
    bind (bind m f) g = bind m (fun a => bind (f a) g)
```

This is the mathematical definition, word for word, in the style of the `Category` class of Chapter 16: two operations, then three fields whose types are the laws. An instance must supply the operations *and* the proofs. `Option` first:

```
/-- `Option` is the monad of computations that may fail without a reason. -/
instance optionMonad : Monad Option where
  pure a := some a
  bind m f :=
    match m with
    | none => none
    | some a => f a
  pure_bind _ _ := rfl
  bind_pure m := by
    cases m <;> rfl
  bind_assoc m _ _ := by
    cases m <;> rfl
```

Left identity holds by `rfl`: `bind (pure a) f` is a `match` on `some a`, which computes to `f a`. The other two laws need a case split on `m`, because a `match` on an unknown `m` cannot compute. `cases m` produces two goals, one for `none` and one for `some a`, and the combinator `<;>` runs the tactic after it, `rfl`, on every goal that `cases` produced. In each case both sides compute to the same value: for `bind_assoc`, both sides are `none` in the first case and `bind (f a) g` in the second. `Except ε` is identical except for the constructor names:

```
/-- `Except ε` is the monad of computations that may fail with an error of
type `ε`: Lean's `Result`. -/
instance exceptMonad (ε : Type) : Monad (Except ε) where
  pure a := .ok a
  bind m f :=
    match m with
    | .error e => .error e
    | .ok a => f a
  pure_bind _ _ := rfl
  bind_pure m := by
    cases m <;> rfl
  bind_assoc m _ _ := by
    cases m <;> rfl
```

Lists need induction. We write `bind` for lists ourselves (it is `flatMap`, the `flat_map` of Rust) and prove the lemma that associativity needs: `bind` distributes over concatenation.

```
/-- `bind` for lists: apply `f` to every element and concatenate the results.
(This is `List.flatMap`, Rust's `flat_map`; we write it out to prove things
about it.) -/
def listBind {α β : Type} : List α → (α → List β) → List β
  | [], _ => []
  | a :: as, f => f a ++ listBind as f

/-- `listBind` distributes over `++`. -/
theorem listBind_append {α β : Type} (xs ys : List α) (f : α → List β) :
    listBind (xs ++ ys) f = listBind xs f ++ listBind ys f := by
  induction xs with
  | nil => rfl
  | cons x xs ih =>
    -- Goal: listBind (x :: xs ++ ys) f = listBind (x :: xs) f ++ listBind ys f
    simp only [List.cons_append, listBind, ih, List.append_assoc]
```

The proof is by induction on `xs`. The empty case holds by computation. In the `cons` case, `simp only` rewrites with four facts: `List.cons_append` moves the head out of the concatenation, `(x :: xs) ++ ys = x :: (xs ++ ys)`; the equations of `listBind` unfold one step on both sides; the induction hypothesis `ih` replaces `listBind (xs ++ ys) f`; and `List.append_assoc` regroups `f x ++ (listBind xs f ++ listBind ys f)` so that both sides become identical. With the lemma, the instance follows:

```
/-- `List` is the monad of nondeterministic computations: a computation returns
every possible result. -/
instance listMonad : Monad List where
  pure a := [a]
  bind := listBind
  pure_bind a f := by
    -- listBind [a] f = f a ++ [] = f a
    simp [listBind]
  bind_pure m := by
    induction m with
    | nil => rfl
    | cons a as ih => simp [listBind, ih]
  bind_assoc m f g := by
    induction m with
    | nil => rfl
    | cons a as ih => simp [listBind, listBind_append, ih]
```

`pure_bind` needs only `simp [listBind]`: `listBind [a] f` unfolds to `f a ++ []`, and `simp` knows that appending `[]` changes nothing. The other two laws go by induction on `m`. Here are the instances at work on the examples of the Rust program:

```
/-- A quantity parsed from text: fails on anything that is not a number. -/
def parseQty (s : String) : Option Nat := s.toNat?

/-- A quantity is accepted only between 1 and 100. -/
def checkQty (n : Nat) : Option Nat := if 1 ≤ n ∧ n ≤ 100 then some n else none

/-- info: (some 12, none, none) -/
#guard_msgs in
#eval (Monad.bind (parseQty "12") checkQty,
       Monad.bind (parseQty "twelve") checkQty,
       Monad.bind (parseQty "500") checkQty)

/-- Every warehouse that has `item`, nondeterministically. -/
def warehouses : String → List String
  | "bolts" => ["Hamburg", "Lyon"]
  | "nuts" => ["Lyon"]
  | _ => []

/-- Every carrier that collects from a warehouse. -/
def carriers : String → List String
  | "Hamburg" => ["DHL", "UPS"]
  | "Lyon" => ["Geodis"]
  | _ => []

/-- info: ["DHL", "UPS", "Geodis"] -/
#guard_msgs in
#eval Monad.bind (warehouses "bolts") carriers
```

Derived operations and their laws

`map`, `join` and Kleisli composition can now be defined once, for every instance of the class:

```
/-- Every monad is a functor: `map f m` runs `m` and applies `f` to the result. -/
def map (f : α → β) (m : M α) : M β :=
```

```
  Monad.bind m (fun a => Monad.pure (f a))

/-- Flatten a computation that returns a computation. -/
def join (mm : M (M α)) : M α :=
  Monad.bind mm (fun m => m)

/-- Kleisli composition: first `f`, then `g`, effects included. (Haskell and
Lean's own library write it `f >=> g`.) -/
def kcomp (f : α → M β) (g : β → M γ) : α → M γ :=
  fun a => Monad.bind (f a) g
```

The claims of the first part of the chapter become theorems. The functor laws of Chapter 17 hold for `map`:

```
/-- The functor laws of Chapter 17 follow from the monad laws. -/
theorem map_id (m : M α) : map (fun a => a) m = m :=
  Monad.bind_pure m

theorem map_comp (f : α → β) (g : β → γ) (m : M α) :
    map (fun a => g (f a)) m = map g (map f m) := by
  -- Unfold `map`; the right side is a bind of a bind.
  simp only [map]
  -- Reassociate, then the inner `bind (pure (f a)) _` is a left identity.
  rw [Monad.bind_assoc]
  simp only [Monad.pure_bind]
```

`map_id` is right identity in disguise: `map (fun a => a) m` *is* `bind m (fun a => pure a)`, which Lean accepts as `bind m pure`, and the proof term is the law itself. For `map_comp`, `simp only [map]` unfolds `map` everywhere, `rw [Monad.bind_assoc]` reassociates the double `bind` on the right-hand side, and the inner `bind (pure (f a)) _` that appears is removed by left identity. The equivalence of the `bind` and `join` presentations, and the three laws of `join` from the diagram above, are proved the same way:

```
/-- `bind` is `map` followed by `join`: the categorical definition of a monad
uses `pure`, `map` and `join` instead of `pure` and `bind`. -/
theorem bind_eq_join_map (m : M α) (f : α → M β) :
    Monad.bind m f = join (map f m) := by
  -- Right side: bind (bind m (fun a => pure (f a))) (fun m => m)
  simp only [join, map]
  -- Reassociate: bind m (fun a => bind (pure (f a)) (fun m => m))
  rw [Monad.bind_assoc]
  -- Left identity turns the inner bind into f a.
  simp only [Monad.pure_bind]

/-- Unit laws for `join`: wrapping with `pure` outside or inside, then
flattening, gives back what we started with. -/
theorem join_pure (m : M α) : join (Monad.pure m) = m :=
  Monad.pure_bind m (fun m => m)

theorem join_map_pure (m : M α) : join (map Monad.pure m) = m := by
  rw [← bind_eq_join_map, Monad.bind_pure]

/-- Associativity of `join`: a three-layer computation can be flattened from
the outside or from the inside. -/
theorem join_join (mmm : M (M (M α))) : join (join mmm) = join (map join mmm) := by
  simp only [join, map]
  -- Reassociate the outer bind on each side.
  rw [Monad.bind_assoc, Monad.bind_assoc]
  -- On the right, `bind (pure (join mm)) id` is `join mm` by left identity.
  simp only [Monad.pure_bind]
```

`join_map_pure` shows a useful trick: rewriting *backwards* with `← bind_eq_join_map` turns `join (map pure m)` into `bind m pure`, and then right identity finishes. Finally, the three laws restated for Kleisli composition:

```
/-- The three monad laws, restated for Kleisli composition: `pure` is a left
and a right identity, and `kcomp` is associative. -/
theorem pure_kcomp (f : α → M β) : kcomp Monad.pure f = f := by
  funext a
  exact Monad.pure_bind a f

theorem kcomp_pure (f : α → M β) : kcomp f Monad.pure = f := by
  funext a
  exact Monad.bind_pure (f a)

theorem kcomp_assoc (f : α → M β) (g : β → M γ) (h : γ → M δ) :
    kcomp (kcomp f g) h = kcomp f (kcomp g h) := by
  funext a
  exact Monad.bind_assoc (f a) g h
```

Each statement is an equation between functions, so the proof starts with `funext a` (two functions are equal if they agree on every argument, as in Chapter 16) and finishes with the corresponding monad law applied at `a`. The library's lawful monads are instances of our class too:

```
/-- Every monad of Lean's library that comes with a `LawfulMonad` instance
gives an instance of our class: the laws are already proved there. -/
abbrev Monad.ofLawful (M : Type → Type) [_root_.Monad M] [LawfulMonad M] : Monad M where
  pure := Pure.pure
  bind := Bind.bind
  pure_bind := LawfulMonad.pure_bind
  bind_pure := _root_.bind_pure
  bind_assoc := LawfulMonad.bind_assoc
```

(`abbrev` declares a definition that Lean unfolds eagerly, as instance-like definitions should be.) It is not an `instance`, so that it does not compete with the instances above; use it as `Monad.ofLawful (StateM Nat)`.

State, Reader and Writer

The function-shaped monads are in `code/lean/Book/Ch20/State.lean`. The state monad first:

```
/-- A computation with a state of type `σ`: given the current state, it returns
a result and the new state. -/
def State (σ α : Type) : Type := σ → α × σ

/-- Run a stateful computation from an initial state. -/
def State.run {σ α : Type} (m : State σ α) (s : σ) : α × σ := m s
```

```
instance stateMonad (σ : Type) : Monad (State σ) where
  pure a := fun s => (a, s)
  bind m f := fun s =>
    let (a, s') := m s
    f a s'
  -- All three laws hold by computation: both sides of each equation unfold to
  -- the same function.
  pure_bind _ _ := rfl
  bind_pure _ := rfl
  bind_assoc _ _ _ := rfl
```

All three laws are proved by `rfl`. That may look like cheating, so take associativity: both sides are functions of a state `s` that run `m` from `s`, then `f` from the resulting state, then `g` from the state after that; Lean unfolds the definitions and finds the same function. For right identity Lean also uses the *eta rule for pairs*: a pair `p` is definitionally equal to `(p.1, p.2)`. (The Rust `State` could not even be tested with `assert_eq!`: closures cannot be compared.)

Two primitive operations read and replace the state, and they combine into `fresh`, which hands out numbers from a counter. With it, the labeling program of the Rust example:

```
/-- Read the state. -/
def get {σ : Type} : State σ σ := fun s => (s, s)

/-- Replace the state. -/
def set {σ : Type} (s : σ) : State σ Unit := fun _ => ((), s)

/-- Return a fresh number from a counter and advance the counter. -/
def fresh : State Nat Nat :=
  Monad.bind get (fun n => Monad.bind (set (n + 1)) (fun _ => Monad.pure n))
```

```
/-- Give every order line a fresh ticket number, in order. -/
def label : List String → State Nat (List (Nat × String))
  | [] => Monad.pure []
  | x :: xs =>
    Monad.bind fresh (fun n =>
      Monad.bind (label xs) (fun rest =>
        Monad.pure ((n, x) :: rest)))

/-- info: ([(100, "bolts"), (101, "nuts"), (102, "washers")], 103) -/
#guard_msgs in
#eval (label ["bolts", "nuts", "washers"]).run 100
```

The Rust program checked the counter for one input; in Lean we prove what it is for every input:

```
/-- Labeling `xs` advances the counter by exactly `xs.length`, whatever the
starting value. -/
theorem label_counter (xs : List String) (n : Nat) :
    ((label xs).run n).2 = n + xs.length := by
  induction xs generalizing n with
  | nil => rfl
  | cons x xs ih =>
    -- Running `label (x :: xs)` from `n` runs `label xs` from `n + 1`.
    show ((label xs).run (n + 1)).2 = n + (x :: xs).length
    rw [ih (n + 1), List.length_cons]
    omega
```

The induction is on the list, and `generalizing n` makes the induction hypothesis hold for *every* starting value of the counter (we need it for `n + 1`, not for `n`). In the `cons` case, `show` replaces the goal by an equivalent one that Lean can check by computation: running `label (x :: xs)` from `n` runs `fresh` (which moves the counter to `n + 1`) and then `label xs` from `n + 1`, and the final `pure` leaves the counter alone. Then the induction hypothesis at `n + 1` and `List.length_cons` turn the goal into `n + 1 + xs.length = n + (xs.length + 1)`, which `omega` closes.

The reader and writer monads complete the family:

```
/-- A computation that reads an environment of type `ρ`. -/
def Reader (ρ α : Type) : Type := ρ → α

instance readerMonad (ρ : Type) : Monad (Reader ρ) where
  pure a := fun _ => a
  bind m f := fun r => f (m r) r
  pure_bind _ _ := rfl
  bind_pure _ := rfl
  bind_assoc _ _ _ := rfl

/-- Ask for the environment. -/
def ask {ρ : Type} : Reader ρ ρ := fun r => r

/-- Pricing parameters, read by every step but changed by none. -/
```

```
structure Pricing where
  unitPrice : Nat
  shipping : Nat

def subtotal (qty : Nat) : Reader Pricing Nat :=
  Monad.bind ask (fun p => Monad.pure (qty * p.unitPrice))

def total (qty : Nat) : Reader Pricing Nat :=
  Monad.bind (subtotal qty) (fun s =>
    Monad.bind ask (fun p => Monad.pure (s + p.shipping)))

/-- info: 85 -/
#guard_msgs in
#eval total 3 { unitPrice := 25, shipping := 10 }
```

```
/-- A computation that also produces a log. -/
def Writer (α : Type) : Type := α × List String

instance writerMonad : Monad Writer where
  pure a := (a, [])
  bind m f :=
    let (b, w) := f m.1
    (b, m.2 ++ w)
  -- Each law of the monad is a law of the log: `[]` is a unit for `++`,
  -- and `++` is associative.
  pure_bind a f := by
    show ((f a).1, [] ++ (f a).2) = f a
    rw [List.nil_append]
    -- Goal: ((f a).1, (f a).2) = f a, true by the eta rule for pairs.
    rfl
  bind_pure m := by
    show (m.1, m.2 ++ []) = m
    rw [List.append_nil]
    rfl
  bind_assoc m f g := by
    show ((g (f m.1).1).1, (m.2 ++ (f m.1).2) ++ (g (f m.1).1).2)
       = ((g (f m.1).1).1, m.2 ++ ((f m.1).2 ++ (g (f m.1).1).2))
    rw [List.append_assoc]

/-- Add a line to the log. -/
def tell (line : String) : Writer Unit := ((), [line])

def reserveLogged (item : String) (qty : Nat) : Writer Nat :=
  Monad.bind (tell s!"reserve {qty} x {item}") (fun _ => Monad.pure qty)

/-- info: (5, ["reserve 2 x bolts", "reserve 3 x nuts"]) -/
#guard_msgs in
#eval Monad.bind (reserveLogged "bolts" 2) (fun a =>
        Monad.bind (reserveLogged "nuts" 3) (fun b => Monad.pure (a + b)))
```

The reader laws hold by `rfl`. The writer laws are more instructive. In each proof, `show` spells out what the two sides compute to, and what remains is a fact about the log: `[] ++ w = w`, `w ++ [] = w`, and the associativity of `++` (the final `rfl` uses the eta rule for pairs). The three monad laws are the three monoid laws of the log. Any monoid would do as a log type (Mathlib's `WriterT` accepts one), and this is the "monoid in the category of endofunctors" slogan in miniature.

20.8 The Kleisli category

Now we build the category of effectful functions on the book's own `Category` class, in `code/lean/Book/Ch20/Kleisli.lean`, which imports `Book.Ch16.Category`. As with the opposite category

of Chapter 16, the objects are wrapped in a structure, so that this category does not clash with `typeCategory`:

```
/-- An object of the Kleisli category of `M`: a Lean type. The wrapper keeps
this category apart from `typeCategory`, whose objects are also types. -/
structure Kleisli (M : Type → Type) where
  /-- The underlying type. -/
  base : Type
```

```
/-- The Kleisli category of a monad: a morphism from `a` to `b` is an effectful
function `a.base → M b.base`, the identity is `pure` and composition is
Kleisli composition. -/
instance kleisliCategory (M : Type → Type) [Monad M] : Category (Kleisli M) where
  Hom a b := a.base → M b.base
  id _ := Monad.pure
  comp g f := kcomp f g
  -- `id ⊚ f = f`: binding into `pure` changes nothing (right identity).
  id_comp f := kcomp_pure f
  -- `f ⊚ id = f`: binding a pure value is application (left identity).
  comp_id f := pure_kcomp f
  -- `h ⊚ (g ⊚ f) = (h ⊚ g) ⊚ f`: associativity of bind.
  assoc h g f := kcomp_assoc f g h
```

A morphism from `a` to `b` is a Kleisli arrow `a.base → M b.base`, the identity is `pure`, and `comp g f` (first `f`, then `g`, in the order of Chapter 16) is `kcomp f g`. Each category law is discharged by one of the Kleisli-form theorems, and each of those came from one monad law. Two of these categories are worth naming:

```
/-- The Kleisli category of `Option`: types and partial functions. -/
abbrev Partial := Kleisli Option

def text : Partial := ⟨String⟩
def qty : Partial := ⟨Nat⟩

/-- Two partial functions, as morphisms. -/
def parseM : text → qty := parseQty
def checkM : qty → qty := checkQty

/-- Their composite, written with the category's `⊚`: first parse, then check. -/
def readQtyM : text → qty := checkM ⊚ parseM

/-- info: [some 12, none, none] -/
#guard_msgs in
#eval ["12", "twelve", "500"].map readQtyM

/-- The Kleisli category of `List`: types and nondeterministic functions,
which are relations. -/
def item : Kleisli List := ⟨String⟩
def site : Kleisli List := ⟨String⟩

def stockedAt : item → site := warehouses
def collectedBy : site → site := carriers

/-- info: ["DHL", "UPS", "Geodis"] -/
#guard_msgs in
#eval (collectedBy ⊚ stockedAt) "bolts"
```

The Kleisli category of `Option` is the category of types and *partial functions*, and `checkM ⊚ parseM` is the composite that Rust's `compose_option` built by hand. The Kleisli category of `List` is the category of types and *nondeterministic functions*, which are relations: `stockedAt` relates an item to every warehouse that has it. (`⟨String⟩` is the anonymous constructor notation, building an object whose `base` is `String`. The type `text → qty` unfolds to `String → Option Nat`, so `parseQty` itself is the morphism.)

The instance shows that the monad laws *imply* the category laws. They are in fact equivalent, and to state the converse we need the operations without the laws:

```
/-- The operations of a monad without the laws. -/
structure MonadOps (M : Type → Type) where
  pure : {α : Type} → α → M α
  bind : {α β : Type} → M α → (α → M β) → M β

/-- Kleisli composition, defined from the raw operations. -/
def MonadOps.kcomp {M : Type → Type} (ops : MonadOps M) {α β γ : Type}
    (f : α → M β) (g : β → M γ) : α → M γ :=
  fun a => ops.bind (f a) g

/-- The monad laws, stated with `bind`. -/
def MonadOps.BindLaws {M : Type → Type} (ops : MonadOps M) : Prop :=
  (∀ {α β : Type} (a : α) (f : α → M β), ops.bind (ops.pure a) f = f a) ∧
  (∀ {α : Type} (m : M α), ops.bind m ops.pure = m) ∧
  (∀ {α β γ : Type} (m : M α) (f : α → M β) (g : β → M γ),
    ops.bind (ops.bind m f) g = ops.bind m (fun a => ops.bind (f a) g))

/-- The category laws of the Kleisli category, stated with `kcomp`. -/
def MonadOps.KleisliLaws {M : Type → Type} (ops : MonadOps M) : Prop :=
  (∀ {α β : Type} (f : α → M β), ops.kcomp ops.pure f = f) ∧
  (∀ {α β : Type} (f : α → M β), ops.kcomp f ops.pure = f) ∧
  (∀ {α β γ δ : Type} (f : α → M β) (g : β → M γ) (h : γ → M δ),
    ops.kcomp (ops.kcomp f g) h = ops.kcomp f (ops.kcomp g h))
```

```
/-- The monad laws hold if and only if Kleisli composition satisfies the laws of
a category. -/
theorem bindLaws_iff_kleisliLaws {M : Type → Type} (ops : MonadOps M) :
    ops.BindLaws ↔ ops.KleisliLaws := by
  constructor
  · -- From bind laws to category laws: apply each law pointwise.
    intro ⟨pure_bind, bind_pure, bind_assoc⟩
    refine ⟨?_, ?_, ?_⟩
    · intro α β f
      funext a
      exact pure_bind a f
    · intro α β f
      funext a
      exact bind_pure (f a)
    · intro α β γ δ f g h
      funext a
      exact bind_assoc (f a) g h
  · -- From category laws to bind laws: a computation `m : M α` is the
    -- Kleisli arrow `fun () => m` applied to `()`.
    intro ⟨pure_kcomp, kcomp_pure, kcomp_assoc⟩
    refine ⟨?_, ?_, ?_⟩
    · intro α β a f
      exact congrFun (pure_kcomp f) a
    · intro α m
      exact congrFun (kcomp_pure (fun (_ : Unit) => m)) ()
    · intro α β γ m f g
      exact congrFun (kcomp_assoc (fun (_ : Unit) => m) f g) ()
```

`constructor` splits the `↔` into two implications. In the first, the destructuring `intro ⟨pure_bind, bind_pure, bind_assoc⟩` names the three bind laws, `refine ⟨?_, ?_, ?_⟩` leaves one goal for each Kleisli law, and each goal is the corresponding bind law applied pointwise after `funext`. The converse contains the one real idea. A Kleisli law talks about functions, but a bind law talks about a computation `m : M α`. The trick is to see `m` as a Kleisli arrow out of the unit type, `fun () => m`, applied to `()`. Then `ops.bind m f` is `ops.kcomp (fun () => m) f ()` by definition, and every bind law is a Kleisli law applied at `()`. `congrFun h x` turns an equation between functions `h : g₁ = g₂` into

the equation `g₁ x = g₂ x` between their values. (The left identity case does not even need the trick: `congrFun` at `a` suffices.)

20.9 Putting it together: an order workflow

Real programs have more than one effect. An order-processing step may fail (an item is unknown or out of stock) *and* it updates a stock (state). Lean combines effects with **monad transformers**: a transformer takes a monad `m` and adds one effect to it. The code is in `code/lean/Book/Ch20/Transformers.lean`, and the definitions are short enough to state as equations that Lean checks by `rfl`:

```
example (σ α : Type) (m : Type → Type) : StateT σ m α = (σ → m (α × σ)) := rfl
example (ε α : Type) (m : Type → Type) : ExceptT ε m α = m (Except ε α) := rfl
example (ρ α : Type) (m : Type → Type) : ReaderT ρ m α = (ρ → m α) := rfl
example (σ α : Type) : StateM σ α = StateT σ Id α := rfl
```

`StateT σ m` is the state monad with `m`'s effect on each step: a function from a state to an `m`-computation of a result and a new state. `ExceptT ε m` is an `m`-computation of a result that may be an error. `ReaderT ρ m` adds an environment. `StateM σ`, the plain state monad, is `StateT σ` over `Id`, the monad with no effect at all.

One workflow, written once

The domain is a small warehouse:

```
/-- What can go wrong with an order. -/
inductive OrderError where
  | emptyOrder
  | unknownItem (item : String)
  | outOfStock (item : String) (wanted available : Nat)
  deriving Repr

/-- The stock of a warehouse: item names and the quantities on hand. -/
abbrev Stock := List (String × Nat)

/-- Set the quantity of `item` to `n`. -/
def Stock.update (stock : Stock) (item : String) (n : Nat) : Stock :=
  stock.map fun (i, q) => if i == item then (i, n) else (i, q)
```

The workflow does not commit to a particular stack of transformers. It is written against two classes: `MonadStateOf Stock m` ("`m` has a state of type `Stock`", providing `get` and `set`) and `MonadExceptOf OrderError m` ("`m` can fail with an `OrderError`", providing `throw` and `try ... catch`). They play the role of trait bounds, in the dictionary-passing sense of Chapter 14:

```
/-- Take `qty` units of `item` out of the stock. The function works in every
monad `m` that has a state of type `Stock` and errors of type `OrderError`. -/
def reserve {m : Type → Type} [Monad m] [MonadStateOf Stock m]
    [MonadExceptOf OrderError m] (item : String) (qty : Nat) : m Unit := do
  let stock ← get
  match stock.lookup item with
  | none => throw (.unknownItem item)
  | some available =>
    if qty ≤ available then
      set (stock.update item (available - qty))
    else
      throw (.outOfStock item qty available)
```

```
/-- Reserve every line of an order; return the number of units. -/
def placeOrder {m : Type → Type} [Monad m] [MonadStateOf Stock m]
    [MonadExceptOf OrderError m] (lines : List (String × Nat)) : m Nat := do
  if lines.isEmpty then
    throw .emptyOrder
  for (item, qty) in lines do
    reserve item qty
  return (lines.map (·.2)).sum
```

(`List.lookup` returns the quantity paired with an item, if there is one, and `(·.2)` is a function that takes the second component of a pair: the `·` marks the argument.) Now there are two natural ways to stack the effects:

```
/-- State outside, errors inside: a failed run returns no state at all. -/
abbrev AllOrNothing := StateT Stock (Except OrderError)

/-- Errors outside, state inside: a failed run still returns the state. -/
abbrev KeepGoing := ExceptT OrderError (StateM Stock)

example (α : Type) : AllOrNothing α = (Stock → Except OrderError (α × Stock)) := rfl
example (α : Type) : KeepGoing α = (Stock → Except OrderError α × Stock) := rfl
```

The `rfl` equations show what the two stacks *are*. In `AllOrNothing`, a run returns either an error, or a result with the new stock. In `KeepGoing`, a run always returns a stock, with either an error or a result. The types already tell you what happens when an order fails halfway, and the runs confirm it:

```
def warehouse : Stock := [("bolts", 10), ("nuts", 4)]

def good : List (String × Nat) := [("bolts", 3), ("nuts", 2)]
def bad : List (String × Nat) := [("bolts", 3), ("nuts", 9)]

/-- info: Except.ok (5, [("bolts", 7), ("nuts", 2)]) -/
#guard_msgs in
#eval (placeOrder good : AllOrNothing Nat).run warehouse

/-- info: Except.error (Ch20.Shop.OrderError.outOfStock "nuts" 9 4) -/
#guard_msgs in
#eval (placeOrder bad : AllOrNothing Nat).run warehouse

/-- info: (Except.ok 5, [("bolts", 7), ("nuts", 2)]) -/
#guard_msgs in
#eval (placeOrder good : KeepGoing Nat).run.run warehouse

/--
info: (Except.error (Ch20.Shop.OrderError.outOfStock "nuts" 9 4), [("bolts", 7), ("nuts",
4)])
-/
#guard_msgs in
#eval (placeOrder bad : KeepGoing Nat).run.run warehouse
```

The good order gives the same result in both stacks. The bad order reserves three bolts, then fails on the nuts. In `AllOrNothing` the failure discards the state, and the reservation of the bolts with it. In `KeepGoing` the caller gets the error *and* a stock in which the bolts are reserved, though the order failed. Transformers do not commute: the order of the stack is a design decision about what a failure means.

If you want `KeepGoing`'s shape but all-or-nothing behavior, restore the stock yourself:

```
/-- Run `action`; if it fails, put the stock back the way it was and fail with
the same error. -/
def atomically {m : Type → Type} {α : Type} [Monad m] [MonadStateOf Stock m]
```

```
    [MonadExceptOf OrderError m] (action : m α) : m α := do
  let saved ← get
  try
    action
  catch e =>
    set saved
    throw e

/--
info: (Except.error (Ch20.Shop.OrderError.outOfStock "nuts" 9 4), [("bolts", 10), ("nuts",
4)])
-/
#guard_msgs in
#eval (atomically (placeOrder bad) : KeepGoing Nat).run.run warehouse
```

`try ... catch e => ...` runs the handler if the action throws; the handler puts the saved stock back and rethrows. The bad order now leaves ten bolts in the warehouse.

Finally, code written for a smaller monad can be used in a bigger one. `checkLines` lives in plain `Except OrderError`, and `placeChecked` uses it inside `AllOrNothing` without any conversion:

```
/-- A check that needs only errors, written in the plain `Except` monad. -/
def checkLines (lines : List (String × Nat)) : Except OrderError Unit :=
  if lines.isEmpty then throw .emptyOrder else pure ()

/-- Inside the stack, the `Except` computation is lifted automatically. -/
def placeChecked (lines : List (String × Nat)) : AllOrNothing Nat := do
  checkLines lines
  for (item, qty) in lines do
    reserve item qty
  return (lines.map (·.2)).sum

/-- info: Except.error (Ch20.Shop.OrderError.emptyOrder) -/
#guard_msgs in
#eval (placeChecked []).run warehouse
```

Lean inserts the translation automatically, through instances of a class `MonadLift` that says how to run an `m`-computation inside an `n`-computation. This is the automatic version of Rust's `.ok_or(...)?`.

The same workflow in Rust

In Rust, the stock is a `&mut` borrow and failure is a `Result` with `?` (`code/rust/examples/ch20_order_workflow.rs`):

```
#[derive(Debug, PartialEq)]
enum OrderError {
    EmptyOrder,
    UnknownItem(String),
    OutOfStock {
        item: String,
        wanted: u32,
        available: u32,
    },
}

/// Item names and quantities on hand. A `BTreeMap` prints in a fixed order.
type Stock = BTreeMap<String, u32>;
```

```
/// Take `qty` units of `item` out of the stock.
fn reserve(stock: &mut Stock, item: &str, qty: u32) -> Result<(), OrderError> {
    let available = stock
        .get_mut(item)
        .ok_or_else(|| OrderError::UnknownItem(item.to_string()))?;
```

```
    if qty > *available {
        return Err(OrderError::OutOfStock {
            item: item.to_string(),
            wanted: qty,
            available: *available,
        });
    }
    *available -= qty;
    Ok(())
}

/// Reserve every line of an order; return the number of units.
fn place_order(stock: &mut Stock, lines: &[(&str, u32)]) -> Result<u32, OrderError> {
    if lines.is_empty() {
        return Err(OrderError::EmptyOrder);
    }
    for &(item, qty) in lines {
        reserve(stock, item, qty)?;
    }
    Ok(lines.iter().map(|&(_, qty)| qty).sum())
}
```

```
/// All or nothing: work on a copy of the stock and commit only on success.
fn place_order_atomically(
    stock: &mut Stock,
    lines: &[(&str, u32)],
) -> Result<u32, OrderError> {
    let mut draft = stock.clone();
    let units = place_order(&mut draft, lines)?;
    *stock = draft;
    Ok(units)
}
```

```
let mut stock = warehouse();
let result = place_order(&mut stock, &good);
println!("plain,  good order: {result:?}, stock {stock:?}");

let mut stock = warehouse();
let result = place_order(&mut stock, &bad);
println!("plain,  bad order : {result:?}, stock {stock:?}");

let mut stock = warehouse();
let result = place_order_atomically(&mut stock, &bad);
println!("atomic, bad order : {result:?}, stock {stock:?}");
```

```
plain,  good order: Ok(5), stock {"bolts": 7, "nuts": 2}
plain,  bad order : Err(OutOfStock { item: "nuts", wanted: 9, available: 4 }), stock
{"bolts": 7, "nuts": 4}
atomic, bad order : Err(OutOfStock { item: "nuts", wanted: 9, available: 4 }), stock
{"bolts": 10, "nuts": 4}
```

The results match Lean's line for line. Plain `place_order` behaves like `KeepGoing`: mutations made through `&mut` before the `?` returned are not undone. `place_order_atomically` behaves like `AllOrNothing`: it works on a copy and commits only after every step succeeded. Rust reaches both semantics with two ordinary features, mutation through a borrow and early return, and you choose by how you write the function. The price is that the type `fn(&mut Stock, ...) -> Result<u32, OrderError>` does not say which semantics you got, while `AllOrNothing` and `KeepGoing` do. That is the trade-off of the whole chapter: Rust's built-in, effect-specific features are cheaper and more flexible; Lean's monads are more explicit and more abstract, and they come with laws you can prove.

20.10 In Mathlib: monads and adjunctions

Mathlib defines monads in the categorical style, in `Mathlib.CategoryTheory.Monad.Basic`: a `CategoryTheory.Monad C` on a category `C` is an endofunctor `T : C ⥤ C` (Chapter 17) with natural transformations `η : 𝟭 C ⟶ T` and `μ : T ⋙ T ⟶ T` (Chapter 18; `𝟭 C` is the identity functor and `⋙` composes functors), satisfying the three equations of our diagram. The file `code/lean/Book/Ch20/InMathlib.lean` states them in a readable form:

```
variable {C : Type*} [Category C] (T : CategoryTheory.Monad C) (X : C)

/-- `T` is an endofunctor: it acts on objects and on morphisms. -/
example : C := T.obj X

/-- The unit `η : 𝟭 C ⟶ T` and the multiplication `μ : T ⋙ T ⟶ T` are natural
transformations (Chapter 18); here are their components at `X`. -/
example : X ⟶ T.obj X := T.η.app X
example : T.obj (T.obj X) ⟶ T.obj X := T.μ.app X

/-- The three laws, in diagrammatic order (`f ≫ g` is "first `f`, then `g`"). -/
example : T.map (T.μ.app X) ≫ T.μ.app X = T.μ.app (T.obj X) ≫ T.μ.app X :=
  T.assoc X
example : T.η.app (T.obj X) ≫ T.μ.app X = 𝟙 (T.obj X) := T.left_unit X
example : T.map (T.η.app X) ≫ T.μ.app X = 𝟙 (T.obj X) := T.right_unit X
```

Each law is a field of the structure (`T.assoc`, `T.left_unit`, `T.right_unit`), written in Mathlib's diagrammatic order, where `f ≫ g` means "first `f`, then `g`" (Chapter 16). A lawful Lean monad gives such a monad on the category `Type`:

```
/-- A lawful Lean monad, seen as a monad on the category `Type`. -/
def optionMonad : CategoryTheory.Monad Type := ofTypeMonad Option

example (α : Type) : optionMonad.obj α = Option α := rfl
```

Adjunctions give monads

Chapter 19 introduced adjunctions through currying: functions $A \times S \to B$ correspond exactly to functions $A \to (S \to B)$, naturally in A and B. In general, an adjunction $L \dashv R$ between functors $L : \mathcal{C} \to \mathcal{D}$ and $R : \mathcal{D} \to \mathcal{C}$ is a natural correspondence between morphisms $L\,A \to B$ in $\mathcal{D}$ and morphisms $A \to R\,B$ in $\mathcal{C}$. Every adjunction yields a monad on $\mathcal{C}$: the functor is the composite $T = R \circ L$, the unit $\eta : A \to R\,(L\,A)$ is what the correspondence assigns to the identity of $L\,A$, and μ is built from the opposite map $L\,(R\,B) \to B$ (the *counit*). Mathlib's `Adjunction.toMonad` performs this construction. Apply it to currying, and out comes a monad you know:

```
/-- The currying adjunction of Chapter 19 on `Type`:
`(S ⊗ A ⟶ B) ≃ (A ⟶ (S ⟶ B))`. -/
example (S : Type) : tensorLeft S ⊣ coyoneda.obj (Opposite.op S) :=
  Types.tensorProductAdjunction S

/-- Every adjunction gives a monad: this one gives the state monad. -/
noncomputable def stateMonad (S : Type) : CategoryTheory.Monad Type :=
  (Types.tensorProductAdjunction S).toMonad

/-- On objects: a computation is a morphism from states to (state, result)
pairs. -/
example (S α : Type) : (stateMonad S).obj α = (S → S ⊗ α) := rfl

/-- The unit pairs the value with the unchanged state: it is `pure`. -/
```

```
example (S α : Type) :
    (stateMonad S).η.app α = ↾(fun (a : α) => ↾(fun (s : S) => (s, a))) := rfl

/-- The multiplication runs the outer computation, then the inner one from the
new state: it is `join`. -/
example (S α : Type) :
    (stateMonad S).μ.app α =
      ↾(fun (k : S → S ⊗ (S → S ⊗ α)) => ↾(fun (s : S) => (k s).2 (k s).1)) := rfl
```

On `Type`, `tensorLeft S` is the functor $A \mapsto S \times A$ (written `S ⊗ A`, the state on the left), and `coyoneda.obj (Opposite.op S)` is the functor $B \mapsto (S \to B)$, written `S → B`. (Mathlib wraps the functions of its category `Type` in a structure; `↾f` turns a plain function into such a morphism.) The monad of the currying adjunction sends A to $S \to S \times A$: the state monad. The `rfl` proofs confirm that its unit is our `pure` and its multiplication is `join` for state: run the outer computation, then the inner computation it returned, from the new state. Its laws come for free from the adjunction.

The construction can be reversed: every monad arises from an adjunction, in fact from two canonical ones:

```
/-- Conversely, every monad comes from an adjunction: the free-algebra functor
is left adjoint to the forgetful functor of its Eilenberg–Moore category ... -/
example : T.free ⊣ T.forget := T.adj
/-- ... and the monad of that adjunction is `T` again, up to isomorphism. -/
example : T.adj.toMonad ≅ T := Adjunction.adjToMonadIso T
/-- The Kleisli category gives another such adjunction. -/
example : Kleisli.Adjunction.toKleisli T ⊣ Kleisli.Adjunction.fromKleisli T :=
  Kleisli.Adjunction.adj T
```

The first uses the *Eilenberg–Moore category* of `T`, whose objects are `T`-algebras (objects `X` with a map `T X → X` compatible with `η` and `μ`), and `adjToMonadIso` says that its monad is `T` again. The second uses the Kleisli category, and you can understand it right now: the left adjoint sends a type A to A as an object of the Kleisli category, the right adjoint sends A to $M\,A$, and the correspondence between Kleisli morphisms $A \to B$ and plain functions $A \to M\,B$ is the identity, because that is how Kleisli morphisms were defined. Monads and adjunctions are two sides of one idea.

20.11 Common confusions

"**A monad is a container.**" `Option` and `Vec` look like containers, and the metaphor helps for them. It fails for `State`, `Reader`, `IO` and futures: a state computation contains no value at all until you run it from some state. A monad is a way of *sequencing computations* that have a certain kind of effect; a container is one special case.

"**Monads make Lean impure.**" The opposite. A `State σ α` is a function, an `IO α` is a value that describes interactions, and every Lean function is still a mathematical function; the runtime runs `IO` actions outside the logic. In Rust, effects happen when the code runs, and the type `fn() -> u32` does not tell you whether the function prints, reads a file or panics.

"**`?` is just `and_then`.**" It is `bind` for the rest of the function body, plus an error conversion through `From`, plus an early `return`. That is why `?` inside a closure returns from the *closure*, not from the

function around it, and why one function body can use `?` for only one monad, the one named by its return type (error E0277).

"**The compiler checks the monad laws.**" Neither Rust nor Lean's `Monad` class does. Lean checks them only where you ask for it: a `LawfulMonad` instance, or a class like ours, requires proofs. Lean's `do` elaborator nonetheless assumes right identity, so an unlawful instance can behave differently from what its `do` blocks seem to say.

"**Monad transformers can be stacked in any order.**" They cannot: the order-processing example shows that `StateT σ (Except ε)` and `ExceptT ε (StateM σ)` give different answers to "what happens to the state when a step fails?".

20.12 Summary

- An effectful function from A to B is a Kleisli arrow $A \to M\,B$, where the type constructor M describes the effect: failure, errors, nondeterminism, state, environment, logging, input and output, suspension.
- A monad is a type constructor with `pure : A → M A` and `bind : M A → (A → M B) → M B` satisfying left identity, right identity and associativity. The laws say that inlining and extracting steps are safe refactorings.
- Equivalently, a monad is a functor with natural transformations η (`pure`) and μ (`join`) satisfying unit and associativity laws, with `bind` $=$ `map` then `join`. Equivalently again, Kleisli arrows form a category with `pure` as identity: the monad laws *are* the category laws, as proved in Lean in both directions.
- Rust uses monads constantly: `?` is `bind` for `Option` and `Result` (plus a `From` conversion), `and_then` and `flat_map` are `bind`, and `.await` is `bind` for futures, compiled into a state machine.
- Rust has no `Monad` trait: there are no higher-kinded types (a GAT family trait works but needs turbofishes), and `bind` must choose between `FnOnce` (which `Vec` cannot implement) and `FnMut` (which costs `Option` users and forces `Clone` bounds). Rust handles effects with dedicated syntax, `&mut` and parameters instead.
- Lean has `Monad`, `do` notation that desugars into nested `bind`s, and `LawfulMonad` for the laws. `IO` is a state-and-exception monad over a token that stands for the world.
- Our own class carries the laws as fields; the instances for `Option`, `Except`, `List`, `State`, `Reader` and `Writer` prove them, and the Kleisli category of any instance is an instance of Chapter 16's `Category`.
- Monad transformers (`StateT`, `ExceptT`, `ReaderT`) combine effects, and the order of the stack matters. Rust gets the same two behaviors from `&mut` and `?`, but its types do not say which one a function has.
- Every adjunction gives a monad (currying gives the state monad), and every monad comes from an adjunction (Eilenberg–Moore, Kleisli).

20.13 Exercises

1. **A counting monad (Lean).** Define `Counted α := α × Nat`, a writer monad whose log is the number of steps performed, as an instance of the class in `Monad.lean` (`pure a := (a, 0)`, and `bind` adds the counts). Prove the three laws, then define `tick : Counted Unit` and check with `#eval` that a three-step computation reports a count of 3. *Hint:* copy `writerMonad` and replace `[]` and `++` with `0` and `+`; the lemmas you need are `Nat.zero_add`, `Nat.add_zero` and `Nat.add_assoc`.
2. **Naturality of `pure` and `join` (Lean).** For an arbitrary instance of our class, prove `map f (Monad.pure a) = Monad.pure (f a)` and `map f (join mm) = join (map (map f) mm)`. These say that η and μ are natural transformations (Chapter 18). *Hint:* the first is one `simp only [map, Monad.pure_bind]`. For the second, unfold `map` and `join`, reassociate with `rw [Monad.bind_assoc]` (as often as needed), and use `Monad.pure_bind` to remove the `pure` steps.
3. **Writer in Rust.** Write `struct Writer<A> { value: A, log: Vec<String> }` with `pure`, `bind` and a `tell` operation, and check the three laws with `assert_eq!` on a few samples. Can your `bind` take an `FnOnce`? Why is that compatible with the discussion of `FnOnce` and `FnMut` in this chapter? *Hint:* `bind` calls the continuation exactly once, as for `Option`; derive `PartialEq` and `Debug` to compare results.
4. **Desugar by hand (Lean, pen and paper).** Write `boxes_result` from `ch20_monads.rs` as a Lean function in `Except String` with a `do` block. Write down, on paper, the term it desugars into, then check your answer with `set_option pp.notation false in #print`. *Hint:* `String.toNat?` returns an `Option`; turn it into an `Except` with a `match`, or with `.elim`. Every `let x ← e` becomes `bind e (fun x => ...)`.
5. **A third layer (Lean).** Add a limit on the number of units per order line to the workflow of `Transformers.lean`, read from an environment with `ReaderT`. Run the good and bad orders in `ReaderT Nat AllOrNothing` with limits 5 and 10, and explain the outputs. *Hint:* inside the new `reserve`, `let limit ← read` gives the environment; `get`, `set` and `throw` are lifted through `ReaderT` automatically; run with `((placeOrder good).run 10).run warehouse`.
6. **The price of `FnMut` (Rust).** Make `compile_fail/ch20_option_bind_fnmut.rs` compile without changing the signature of `bind_option`. Then explain, in terms of Chapter 8's closure traits and Chapter 10's remark that duplicating a value needs `Clone`, why your fix is necessary for an `FnMut` closure but not for an `FnOnce` one. *Hint:* the closure must still own a `String` after it runs; return `customer.clone()`.